

- Technical Report: OCR-Based Legal Document Extraction Pipeline
 - 1. Project Overview
- 2. High-Level Pipeline Flow
- 3. Main Input and Output Folders
 - 3.1 Documents/
 - 3.2 phase1_output/
 - 3.3 phase2_output/
 - 3.4 phase3_output/
- 4. File-by-File Explanation
 - 4.1 pdf_to_image.py
 - Purpose
 - Main Input
 - Main Output
 - Main Functions
 - save_pdf_pages_as_images()
 - calculate_blank_score()
 - calculate_dark_pixel_ratio()
 - detect_rotation_risk()
 - inspect_image()
 - What anyone Can Safely Change
 - When to Use
 - 4.2 ocr.py
 - Purpose
 - Main Input
 - Main Output
 - Important Config Variables
 - Main Functions
 - get_pdf_files()
 - run_chandra_cli()
 - find_chandra_outputs()
 - run_simple_sanity_report()
 - What anyone Can Safely Change
 - When to Use
 - 4.3 page_by_page_ocr.py
 - Purpose
 - Main Input
 - Main Output

- Page JSON Structure
- Important Config Variables
- Main Functions
 - `get_pdf_files()`
 - `get_pdf_page_count()`
 - `run_chandra_single_page()`
 - `compute_sanity()`
 - `run_page_level_ocr()`
 - `run_simple_sanity_report()`
- What anyone Can Safely Change
- Warning
- 4.4 chunking.py
 - Purpose
 - Main Input
 - Main Output
 - Chunk JSON Structure
 - Important Config Variables
 - Main Functions
 - `get_document_folders()`
 - `load_pages_from_doc_folder()`
 - `build_chunks_for_document()`
 - `save_document_chunks()`
 - What anyone Can Safely Change
 - Practical Advice
- 4.5 llm_client.py
 - Purpose
 - Why This File Exists
 - Supported Backends
 - Main Config Object: `LLMRequestConfig`
 - Main Public Function: `chat_completion()`
 - Ollama Backend
 - HuggingFace Backend
 - HuggingFace Model Loading
 - HuggingFace JSON Handling
 - Thinking Block Cleanup
 - What anyone Can Safely Change
 - Dependencies Required by `llm_client.py`
 - When to Modify This File

- 4.6 `stateful_extraction_2.py`
 - Purpose
 - Major Update in Current Version
 - Main Input
 - Main Output
 - Important Config Variables
 - Local LLM Backend Config
 - Important Warning: Chunk Range
 - Extractor Output Schema
 - Allowed Block Types
 - Carry-Forward State
 - Previous Open Point Block
 - Important Rule
 - Main Functions
 - `get_document_jobs()`
 - `build_user_prompt()`
 - `call_model()`
 - `extract_json_object()`
 - `normalize_block()`
 - `normalize_state()`
 - `validate_and_normalize_output()`
 - `fallback_output()`
 - `run_extractor_once()`
 - `process_chunk()`
 - `run_extraction_for_document()`
 - `run_extraction()`
 - What anyone Can Safely Change
 - What Should Be Changed Carefully
- 4.7 `validator.py`
 - Purpose
 - Major Update in Current Version
 - Validator Backend Config
 - Main Input
 - Main Output
 - The Four Major Errors It Checks
 - 1. `wrong_block_split`
 - 2. `copied_context_text`
 - 3. `wrong_block_type`

- 4. carry_state_inconsistency
- Conservative Validation Philosophy
- Main Functions
 - compact_text()
 - normalize_validator_response()
 - parse_json_object()
 - validate_extraction_with_agent()
- What anyone Can Safely Change
- What Should Be Changed Carefully
- 4.8 stitch_points.py
 - Purpose
 - Main Input
 - Main Output
 - Final JSON Structure
 - Important Config Variables
 - How Stitching Works
 - open_groups
 - latest_group_index
 - Main Stitching Cases
 - Case 1: Explicit continuation
 - Case 2: Recovered continuation
 - Case 3: Implicit open-group continuation
 - Case 4: New point
 - Duplicate Boundary Handling
 - Main Functions
 - get_document_jobs()
 - load_phase2_pages()
 - stitch_document()
 - run_phase3()
 - What anyone Can Safely Change
 - What Should Be Changed Carefully
- 5. Recommended Running Procedure
 - Step 1: Put PDF in Documents/
 - Step 2: Run page-level OCR
 - Step 3: Run chunking
 - Step 4: Run Phase 2 extraction
 - Step 5: Run Phase 3 stitching
- 6. Backend Switching Guide

- 6.1 Use HuggingFace for Extractor and Validator
- 6.2 Use Ollama for Extractor and Validator
- 6.3 Use Different Backends for Extractor and Validator
- 7. Dependencies
 - General OCR and Processing Dependencies
 - Chandra OCR
 - Ollama Backend
 - HuggingFace Backend
- 8. Current Limitations and Warnings
 - 8.1 Paragraph extraction is not implemented yet
 - 8.2 CHUNK_START = 48 can skip pages
 - 8.3 HuggingFace does not strictly enforce JSON
 - 8.4 Page numbering is zero-based
 - 8.5 Validator is conservative by design
 - 8.6 Some comments are outdated
- 9. Safe Modification Checklist
 - To process a new PDF
 - To change OCR settings
 - To change chunk context size
 - To change extractor model
 - To change validator model
 - To run the whole document
 - To run only some chunks
 - To disable validator
- 10. Developer Notes
 - 10.1 Schema consistency is critical
 - 10.2 llm_client.py should remain stable
 - 10.3 Full-document run checklist
- 11. Summary for Teammates

Technical Report: OCR-Based Legal Document Extraction Pipeline

1. Project Overview

This project is an OCR-based legal document extraction pipeline. Its purpose is to take legal PDF documents, perform OCR, divide the OCR result into page-based chunks, extract structured legal content using a local LLM, validate the extracted structure, and finally stitch multi-page legal points into a clean final JSON document.

The current project structure is:

```
try2/
├── .vscode/
├── Documents/
├── phase1_output/
├── phase2_output/
├── phase3_output/
├── __pycache__/
├── pdf_to_image.py
├── ocr.py
├── page_by_page_ocr.py
├── chunking.py
├── llm_client.py
├── stateful_extraction_2.py
├── validator.py
└── stitch_points.py
```

The pipeline is organized into three major phases:

```
Phase 1: OCR and page preparation
├── pdf_to_image.py
├── ocr.py
└── page_by_page_ocr.py

Phase 1B: Chunk preparation
└── chunking.py

Phase 2: LLM-based legal structure extraction
├── llm_client.py
├── stateful_extraction_2.py
└── validator.py

Phase 3: Final stitching
└── stitch_points.py
```

The recommended main execution flow is:

```
python page_by_page_ocr.py
python chunking.py
```

```
python stateful_extraction_2.py
python stitch_points.py
```

`pdf_to_image.py` and `ocr.py` are supporting scripts. They are useful for visual checking and full-document OCR, but the main working flow uses page-level OCR through `page_by_page_ocr.py`.

2. High-Level Pipeline Flow

The complete processing flow is:

```
Raw PDF
↓
page_by_page_ocr.py
↓
phase1_output/page_by_page_ocr/without_headers_footers/<document_name>/page_XXXX.js
on
↓
chunking.py
↓
phase1_output/chunks/without_headers_footers/<document_name>/chunks.json
↓
stateful_extraction_2.py
↓
llm_client.py
↓
validator.py
↓
phase2_output/page_outputs_2/<document_name>/<chunk_id>_page_XXXX.json
↓
stitch_points.py
↓
phase3_output/final_documents/<document_name>/final_stitched_document.json
```

The most important idea is that each stage produces files that the next stage expects. If one stage changes its output format, the next stage may break.

3. Main Input and Output Folders

3.1 Documents/

This folder stores the original PDF files.

Example:

```
Documents/20160408_Finance_Act_2013.pdf
```

Most scripts use a configurable `INPUT_PATH`, so anyone can process either one PDF file or a folder of PDFs.

3.2 phase1_output/

This folder stores OCR and chunking outputs.

Important paths:

```
phase1_output/page_by_page_ocr/  
phase1_output/page_by_page_ocr/without_headers_footers/  
phase1_output/chunks/  
phase1_output/chunks/without_headers_footers/  
phase1_output/reports/
```

The current main pipeline usually uses:

```
phase1_output/page_by_page_ocr/without_headers_footers/<document_name>/page_XXXX.js  
on
```

and then:

```
phase1_output/chunks/without_headers_footers/<document_name>/chunks.json
```

3.3 phase2_output/

This folder stores LLM extraction outputs.

Important paths:

```
phase2_output/page_outputs_2/  
phase2_output/reports/
```

For each chunk/page, the extractor saves one JSON file:

```
phase2_output/page_outputs_2/<document_name>/<chunk_id>_page_XXXX.json
```

The Phase 2 report is saved inside:

```
phase2_output/reports/
```

3.4 phase3_output/

This folder stores the final stitched result.

Important paths:

```
phase3_output/final_documents/  
phase3_output/reports/
```

Final stitched document:

```
phase3_output/final_documents/<document_name>/final_stitched_document.json
```

Phase 3 report:

```
phase3_output/reports/<document_name>/phase3_stitch_report.json
```

Master report:

4. File-by-File Explanation

4.1 pdf_to_image.py

Purpose

`pdf_to_image.py` converts a PDF into page images and performs image-level sanity checks.

This script is useful when the OCR quality is poor and we need to visually inspect the original PDF pages. It can help detect:

- blank pages,
- pages with very low text density,
- possible rotated or landscape pages,
- low-resolution pages.

This file is not required for the main extraction flow, but it is useful for debugging OCR problems.

Main Input

Configured at the top of the file:

```
PDF_PATH = r"001-Law-1994 Germany DTAA.pdf"
```

This should be changed to the PDF file that needs image conversion.

Main Output

Images are saved inside:

```
phase1_output/page_images/<PDF_NAME>/
```

Reports are saved inside:

```
phase1_output/reports/
```

The report files are:

```
phase1_image_sanity_report.json  
phase1_image_sanity_report.csv
```

Main Functions

save_pdf_pages_as_images()

Converts each PDF page into an image.

It uses:

```
pdf2image.convert_from_path()
```

Each page is saved as a PNG file.

calculate_blank_score()

Calculates how much of the image is almost white.

A higher blank score means the page may be blank.

calculate_dark_pixel_ratio()

Measures how many pixels are dark.

This gives a rough estimate of how much text or ink exists on the page.

detect_rotation_risk()

Checks whether the image width is greater than its height.

If width is greater than height, the page may be landscape or rotated.

inspect_image()

Collects all image-level information for one page.

It returns:

```
{
  "page_number": 1,
  "image_path": "...",
  "width": 2480,
  "height": 3508,
  "mode": "RGB",
  "blank_score": 0.92,
  "dark_pixel_ratio": 0.04,
  "rotation_risk": false,
  "warnings": [],
  "sanity_status": "ok"
}
```

What anyone Can Safely Change

```
PDF_PATH
DPI
IMAGE_FORMAT
POPPLER_PATH
```

Recommended default:

```
DPI = 300
```

Higher DPI may improve readability but will increase file size and processing time.

When to Use

Use this file when:

- OCR output looks wrong.
 - Pages may be blank or rotated.
 - You want to inspect page images manually.
 - You want a quick image sanity report before OCR.
-

4.2 `ocr.py`

Purpose

`ocr.py` runs Chandra OCR on full PDF documents. It can process either a single PDF or all PDFs inside a folder.

This file generates full-document OCR outputs such as Markdown, HTML, metadata JSON, and extracted images.

However, this is not the best script for the current page-by-page extraction pipeline. The current pipeline is better served by `page_by_page_ocr.py`.

Main Input

Configured at the top:

```
INPUT_PATH = Path("Documents/Finance Acts/")
```

This can be:

```
INPUT_PATH = Path("Documents/example.pdf")
```

or:

```
INPUT_PATH = Path("Documents/Finance Acts/")
```

If a folder is given, it recursively searches for all PDF files inside it.

Main Output

If headers and footers are included:

```
phase1_output/full_document_ocr/
```

If headers and footers are excluded:

```
phase1_output/full_document_ocr/without_headers_footers/
```

Reports are saved inside:

```
phase1_output/reports/ocr/
```

Important Config Variables

```
METHOD = "hf"  
PAGE_RANGE = None  
MAX_OUTPUT_TOKENS = 12384  
INCLUDE_IMAGES = True  
INCLUDE_HEADERS_FOOTERS = True  
TIMEOUT_SECONDS = 60 * 60 * 3
```

Meaning:

- `METHOD = "hf"` uses local HuggingFace mode for Chandra OCR.
- `METHOD = "vllm"` can be used if the Chandra vLLM server is running.
- `PAGE_RANGE = None` processes the full document.
- `INCLUDE_IMAGES = True` keeps images extracted by OCR.
- `INCLUDE_HEADERS_FOOTERS = True` includes headers and footers.
- `TIMEOUT_SECONDS` controls maximum OCR runtime.

Main Functions

`get_pdf_files()`

Finds PDF files from the configured `INPUT_PATH`.

It supports both:

1. a single PDF file,
2. a folder containing multiple PDFs.

`run_chandra_cli()`

Builds and runs the Chandra OCR command.

Example command:

```
chandra <pdf_path> <output_dir> --method hf --max-output-tokens 12384 --include-images --include-headers-footers
```

find_chandra_outputs()

Finds generated Chandra OCR outputs:

```
*.md  
*.html  
*metadata.json  
*.png  
*.jpg  
*.jpeg  
*.webp
```

run_simple_sanity_report()

Creates a summary report showing:

- total PDFs,
- successful PDFs,
- failed PDFs,
- Markdown files found,
- HTML files found,
- metadata JSON files found,
- extracted images found,
- warnings.

What anyone Can Safely Change

```
INPUT_PATH  
METHOD  
PAGE_RANGE  
MAX_OUTPUT_TOKENS
```



```
INCLUDE_IMAGES
INCLUDE_HEADERS_FOOTERS
TIMEOUT_SECONDS
```

When to Use

Use this file when:

- You want full-document OCR output.
- You want the whole document as Markdown or HTML.
- You do not need one JSON file per page.
- You are debugging Chandra OCR output at the document level.

For the main extraction pipeline, use `page_by_page_ocr.py` instead.

4.3 `page_by_page_ocr.py`

Purpose

`page_by_page_ocr.py` is the main OCR script for the pipeline.

It runs Chandra OCR page by page and saves each page as a separate JSON file. This is important because `chunking.py` expects page-level JSON files.

Main Input

Configured at the top:

```
INPUT_PATH = Path(r"Documents\20160408_Finance_Act_2013.pdf")
```

This can be either:

```
INPUT_PATH = Path(r"Documents\example.pdf")
```

or:

```
INPUT_PATH = Path(r"Documents\Finance Acts")
```

If a folder is given, all PDF files inside it are processed.

Main Output

If headers and footers are excluded:

```
phase1_output/page_by_page_ocr/without_headers_footers/<document_name>/
```

If headers and footers are included:

```
phase1_output/page_by_page_ocr/<document_name>/
```

Each page becomes one JSON file:

```
page_0000.json  
page_0001.json  
page_0002.json  
...
```

Important note: the page numbering is currently zero-based. That means the first page is stored as `page_0000.json`.

Page JSON Structure

Each page JSON looks like:

```
{
  "document_id": "20160408_Finance_Act_2013",
  "source_file": "Documents\\20160408_Finance_Act_2013.pdf",
  "page_number": 0,
  "ocr": {
    "engine": "chandra_ocr_2",
    "method": "hf"
  },
  "content": {
    "raw_markdown": "...",
    "raw_html": "..."
  },
  "metadata": {},
  "sanity": {
    "char_count": 1234,
    "line_count": 50,
    "bangla_chars": 800,
    "english_chars": 100,
    "warnings": [],
    "status": "ok"
  }
}
```

The most important field for the next stage is:

```
"content": {
  "raw_markdown": "..."
}
```

`chunking.py` reads `raw_markdown`.

Important Config Variables

```
METHOD = "hf"
MAX_OUTPUT_TOKENS = 12384
TIMEOUT_SECONDS = 60 * 20
INCLUDE_HEADERS_FOOTERS = False
```

The current setup excludes headers and footers:

```
INCLUDE_HEADERS_FOOTERS = False
```

This is usually better for legal structure extraction because repeated page headers and footers can confuse the LLM.

Main Functions

`get_pdf_files()`

Finds PDFs from the configured input path.

`get_pdf_page_count()`

Uses `pypdf.PdfReader` to count how many pages the PDF has.

`run_chandra_single_page()`

Runs Chandra OCR for one page at a time.

It creates a temporary folder:

```
temp_page_XXXX/
```

Then runs a command like:

```
chandra <pdf_path> <temp_dir> --method hf --page-range <page_number> --max-output-tokens 12384 --no-headers-footers
```

After OCR, it reads the generated Markdown, HTML, and metadata files.

`compute_sanity()`

Checks the OCR text for:

- empty text,
 - very short text,
 - encoding noise,
 - missing Bangla or English characters.
-

run_page_level_ocr()

Runs OCR for every page in a PDF and saves the page JSON outputs.

run_simple_sanity_report()

Creates a report containing:

- input path,
 - whether headers/footers were included,
 - total PDFs,
 - total pages,
 - successful pages,
 - failed pages,
 - warning pages,
 - failed pages by PDF.
-

What anyone Can Safely Change

```
INPUT_PATH
METHOD
MAX_OUTPUT_TOKENS
TIMEOUT_SECONDS
INCLUDE_HEADERS_FOOTERS
```

Warning

The code currently loops like this:

```
for page_number in range(0, total_pages):
```

So page numbering starts at 0.

This is fine if Chandra accepts zero-based page numbers in the current environment. If OCR skips or misaligns pages, this should be checked first.

4.4 chunking.py

Purpose

`chunking.py` converts page-level OCR JSON files into LLM-ready chunks.

Each chunk mostly represents one page, but it also includes a small amount of context from the previous and next pages.

This context helps the LLM decide whether a legal point continues across pages.

Main Input

Configured at the top:

```
PAGE_OCR_INPUT =  
Path(r"phase1_output\page_by_page_ocr\without_headers_footers\20160408_Finance_Act_  
2013")
```

This can be:

1. a single document folder containing `page_*.json`, or
 2. a parent folder containing multiple document folders.
-

Main Output

Configured as:

```
OUTPUT_ROOT = Path("phase1_output/chunks/without_headers_footers")
```

For one document:

```
phase1_output/chunks/without_headers_footers/<document_name>/chunks.json
```

Chunk JSON Structure

Each chunk looks like:

```
{
  "document_name": "20160408_Finance_Act_2013",
  "chunk_id": "20160408_Finance_Act_2013_chunk_0001",
  "page_number": 0,
  "text": "current page OCR markdown",
  "previous_tail": "last 800 characters from previous page",
  "next_head": "first 800 characters from next page"
}
```

The field **text** is the actual current page content.

The fields **previous_tail** and **next_head** are only context. The extractor should not copy text from them into the output.

Important Config Variables

```
TAIL_SIZE = 800
HEAD_SIZE = 800
```

Meaning:

- `TAIL_SIZE = 800`: keep the last 800 characters from the previous page.
 - `HEAD_SIZE = 800`: keep the first 800 characters from the next page.
-

Main Functions

`get_document_folders()`

Detects whether the input is one document folder or a parent folder containing multiple document folders.

`load_pages_from_doc_folder()`

Loads all `page_*.json` files and sorts them by page number.

`build_chunks_for_document()`

Creates one chunk per page.

Each chunk includes:

- document name,
 - chunk ID,
 - page number,
 - current page OCR text,
 - previous page tail,
 - next page head.
-

`save_document_chunks()`

Saves all chunks into one file:


```
chunks.json
```

What anyone Can Safely Change

```
PAGE_OCR_INPUT  
OUTPUT_ROOT  
TAIL_SIZE  
HEAD_SIZE
```

Practical Advice

If the extractor fails to detect cross-page continuation, increase **TAIL_SIZE** and **HEAD_SIZE**.

If the extractor starts copying previous or next page text into the current output, reduce **TAIL_SIZE** and **HEAD_SIZE**.

4.5 **llm_client.py**

Purpose

llm_client.py is the shared local LLM backend layer.

This is the newest architectural addition. Earlier, **stateful_extraction_2.py** and **validator.py** directly called Ollama. Now both files call **llm_client.py**, and this file decides whether to use:

```
Ollama local server  
HuggingFace Transformers local model
```

This makes the pipeline cleaner and more flexible.

Why This File Exists

Without `llm_client.py`, every file that needs an LLM would need its own separate Ollama or HuggingFace logic. That creates duplicated code and makes backend switching messy.

With `llm_client.py`, the extractor and validator only need to call:

```
chat_completion(...)
```

and pass a configuration object:

```
LLMRequestConfig(...)
```

This keeps the model-calling logic centralized.

Supported Backends

The supported backend names are:

```
"ollama"  
"hf"  
"huggingface"
```

Internally, the code defines:

```
BackendName = Literal["ollama", "hf", "huggingface"]
```

Main Config Object: `LLMRequestConfig`

The file defines a dataclass:

```
@dataclass(frozen=True)
class LLMRequestConfig:
    backend: BackendName
    model: str
    temperature: float = 0.2
    top_p: float = 0.1
    num_ctx: Optional[int] = None
    max_new_tokens: int = 4096
    think: bool = False
    response_format: str = "json"
```

This object stores all model-calling settings.

Important fields:

- `backend`: whether to use Ollama or HuggingFace.
- `model`: model name.
- `temperature`: controls randomness.
- `top_p`: nucleus sampling parameter.
- `num_ctx`: context length, mainly useful for Ollama.
- `max_new_tokens`: maximum tokens to generate, mainly important for HuggingFace.
- `think`: enables thinking mode if supported.
- `response_format`: usually "json".

Main Public Function: `chat_completion()`

This is the public entry point used by the rest of the pipeline.

Signature:

```
def chat_completion(
    *,
```

```
system_prompt: str,  
user_prompt: str,  
config: LLMRequestConfig,  
) -> str:
```

It receives:

- system prompt,
- user prompt,
- request config.

Then it checks the backend:

```
if backend == "ollama":  
    return _chat_ollama(...)  
  
if backend in {"hf", "huggingface"}:  
    return _chat_huggingface(...)
```

So the extractor and validator do not need to know the low-level details of Ollama or HuggingFace.

Ollama Backend

Function:

```
_chat_ollama()
```

This function:

1. imports the `ollama` Python package,
2. builds an `options` dictionary,
3. passes temperature, top-p, and context length,
4. calls:

```
ollama.chat(...)
```

The response is returned as:

```
response["message"]["content"]
```

Ollama has native JSON formatting support through:

```
format="json"
```

So when `response_format = "json"`, Ollama is instructed to return JSON.

HuggingFace Backend

Function:

```
_chat_huggingface()
```

This function uses:

```
AutoProcessor  
AutoModelForCausalLM
```

It loads the model and processor through:

```
_load_hf_bundle()
```

The model is cached using:

```
@lru_cache(maxsize=2)
```

This is important because loading a HuggingFace model is expensive. With caching, the model is loaded once and reused across extractor and validator calls.

HuggingFace Model Loading

The function `_load_hf_bundle()`:

1. reads an optional HuggingFace token from:

```
HF_TOKEN  
HUGGINGFACE_TOKEN
```

2. loads the processor,
3. loads the model with:

```
device_map="auto"
```

4. uses automatic dtype selection:

```
dtype="auto"
```

or fallback:

```
torch_dtype="auto"
```

5. sets the model to evaluation mode:

```
model.eval()
```

HuggingFace JSON Handling

Unlike Ollama, HuggingFace generation does not enforce JSON through a native `format="json"` parameter.

So if `response_format == "json"`, `llm_client.py` appends an extra instruction to the user prompt:

```
Return exactly one valid JSON object only. No markdown. No explanation. No text before or after JSON.
```

This does not guarantee JSON, but it improves the chance of valid output. The existing parser in `stateful_extraction_2.py` and `validator.py` still handles extra text defensively.

Thinking Block Cleanup

Some models may emit reasoning blocks such as:

```
<think>...</think>
```

or:

```
<thinking>...</thinking>
```

`llm_client.py` includes:

```
_strip_thinking_blocks()
```

to remove these from the final model response before JSON parsing.

What anyone Can Safely Change

Usually, teammates should not change the internal code of `llm_client.py`.

They should change backend settings from the caller files instead:

- `stateful_extraction_2.py` for extractor settings,
- `validator.py` for validator settings.

However, advanced users can change:

```
maxsize=2  
device_map="auto"  
dtype="auto"
```

only if they understand HuggingFace model loading and GPU memory behavior.

Dependencies Required by `llm_client.py`

For Ollama backend:

```
pip install ollama
```

For HuggingFace backend:

```
pip install -U transformers torch accelerate
```

If using a gated HuggingFace model, set one of these environment variables:

```
HF_TOKEN=<your_token>
```

or:

```
HUGGINGFACE_TOKEN=<your_token>
```

When to Modify This File

Modify `llm_client.py` only when:

- adding a new backend,
- changing HuggingFace loading behavior,
- changing how JSON response cleanup works,
- changing how thinking blocks are removed,
- changing shared LLM error handling.

For normal model switching, do not modify this file. Change the config in the caller files instead.

4.6 `stateful_extraction_2.py`

Purpose

`stateful_extraction_2.py` is the main Phase 2 extraction engine.

It reads `chunks.json`, sends each page chunk to a local LLM, extracts structured legal content blocks, optionally validates the output, repairs invalid extraction, and saves one JSON output per page/chunk.

This file is the core intelligence layer of the project.

Major Update in Current Version

The extractor no longer directly calls Ollama.

It now imports:

```
from llm_client import LLMRequestConfig, chat_completion
```

and uses:

```
call_model()
```

to call either:

```
HuggingFace local model  
Ollama local server
```

This makes the extractor backend-independent.

Main Input

Configured at the top:

```
INPUT_CHUNKS_PATH =  
Path(r"phase1_output\chunks\without_headers_footers\20160408_Finance_Act_2013\chunks.json")
```

This can be:

1. a direct `chunks.json` file,
 2. a document folder containing `chunks.json`,
 3. a parent folder containing multiple document folders.
-

Main Output

Page-level extraction outputs are saved inside:

```
phase2_output/page_outputs_2/<document_name>/
```

Each page/chunk produces one JSON file:

```
<chunk_id>_page_XXXX.json
```

Reports are saved inside:

```
phase2_output/reports/
```

Example:

```
phase2_output/reports/20160408_Finance_Act_2013_report.json
```

Important Config Variables

```
INPUT_CHUNKS_PATH = Path(...)
OUTPUT_ROOT = Path("phase2_output")
PAGE_OUTPUT_ROOT = OUTPUT_ROOT / "page_outputs_2"
REPORT_ROOT = OUTPUT_ROOT / "reports"

MAX_RETRIES = 2
STOP_ON_FAILURE = False

USE_VALIDATOR_AGENT = True
MAX_VALIDATOR_REPAIR_ROUNDS = 3

SEND_PREVIOUS_OPEN_POINT_BLOCK = True
PREVIOUS_OPEN_POINT_TEXT_LIMIT = None
```

These control file paths, retry behavior, validator usage, and previous open point context.

Local LLM Backend Config

Current extractor backend config:

```
MODEL_BACKEND = "hf"
HF_MODEL = "google/gemma-4-E4B-it"
OLLAMA_MODEL = "gemma4:latest"
THINK = True
TEMPERATURE = 0.2
TOP_P = 0.1
NUM_CTX = 32768
MAX_NEW_TOKENS = 8192
```

Meaning:

- `MODEL_BACKEND = "hf"` means the extractor currently uses a HuggingFace local model.
- If changed to `"ollama"`, it will use the local Ollama server.
- `HF_MODEL` is used only when backend is `"hf"` or `"huggingface"`.
- `OLLAMA_MODEL` is used only when backend is `"ollama"`.
- `MAX_NEW_TOKENS` is mainly for HuggingFace output length.
- `NUM_CTX` is mainly useful for Ollama context length.
- `THINK` is passed to both backends if supported.

Important Warning: Chunk Range

Current setting:

```
CHUNK_START = 48
CHUNK_END = None
```

This means the extractor starts from chunk 48 and skips earlier chunks.

This is useful for debugging, but dangerous for full-document extraction.

For a full document run, change it to:

```
CHUNK_START = 0
CHUNK_END = None
```

To process only a specific range:

```
CHUNK_START = 10
CHUNK_END = 20
```

This processes chunks 10 through 19.

Extractor Output Schema

The extractor returns this structure:

```
{
  "chunk_id": "",
  "page_number": 0,
  "document_id": "",
  "content_blocks": [
    {
      "block_id": "",
      "type": "point | metadata | ignore | uncertain",
      "point_number": "",
      "text": "",
      "continues_from_previous": false,
      "continues_to_next": false,
      "stitch_group_id": "",
      "stitching_note": ""
    }
  ],
  "output_carry_forward_state": {
    "status": "open | closed | uncertain",
    "active_point_number": "",
    "active_stitch_group_id": "",
    "active_point_summary": "",
    "numbering_system": "",
    "expected_next_main_point": "",
    "continuation_hint": "",
    "last_visible_text": ""
  }
}
```

Allowed Block Types

Current allowed block types are:

```
point
metadata
ignore
uncertain
```

Meaning:

- **point**: main numbered legal content.
- **metadata**: document title, chapter title, date, authority line, preamble, signature block, etc.
- **ignore**: page number, repeated footer, OCR garbage.
- **uncertain**: used when the model is unsure.

Important limitation: the current schema does not yet support a dedicated **paragraph** block type.

Carry-Forward State

The extractor uses carry-forward state to remember legal continuity across pages.

Example:

```
{
  "status": "open",
  "active_point_number": "&",
  "active_stitch_group_id": "point_&",
  "active_point_summary": "The current point discusses...",
  "numbering_system": "bangla_digits",
  "expected_next_main_point": "৬",
  "continuation_hint": "The next page may continue this point.",
  "last_visible_text": "..."
```

If a point continues to the next page:

```
"continues_to_next": true
```

then the carry state should usually be:

```
"status": "open"
```

If there is no active continuation, the carry state should usually be:

```
"status": "closed"
```

Previous Open Point Block

If a point continues to the next page, the extractor can send a compact version of the previous open point to the next chunk.

Controlled by:

```
SEND_PREVIOUS_OPEN_POINT_BLOCK = True  
PREVIOUS_OPEN_POINT_TEXT_LIMIT = None
```

Function:

```
get_previous_open_point_block()
```

This finds the last **point** block where:

```
"continues_to_next": true
```

and sends that point as context to the next extraction call.

This helps maintain the same:

```
point_number  
stitch_group_id
```

across pages.

Important Rule

The extractor must extract text only from:

```
current_page_text
```

It must not copy text from:

```
previous_tail  
next_head  
previous_open_point_block
```

Those fields are only for understanding continuation.

Main Functions

get_document_jobs()

Detects whether the input is:

- a direct `chunks.json`,
 - a document folder containing `chunks.json`,
 - a parent folder containing multiple document folders.
-

build_user_prompt()

Builds the JSON payload sent to the LLM.

It includes:

- chunk ID,
- page number,
- document ID,
- previous state,

- previous tail,
 - current page text,
 - next head,
 - previous open point block,
 - validator instruction if this is a repair attempt.
-

call_model()

This is the new backend-independent model caller.

It selects the correct model name:

```
model_name = HF_MODEL if MODEL_BACKEND in ["hf", "huggingface"] else OLLAMA_MODEL
```

Then calls:

```
chat_completion(...)
```

with an `LLMRequestConfig`.

This is where `stateful_extraction_2.py` connects to `llm_client.py`.

extract_json_object()

Parses model output into JSON.

It first tries direct `json.loads()`. If that fails, it extracts the largest JSON object from the text.

This is important because local models may sometimes return extra text even when instructed not to.

normalize_block()

Normalizes each extracted block.

It ensures:

- valid block type,
 - valid boolean fields,
 - stable block ID,
 - clean text,
 - empty point fields for non-point blocks,
 - auto-created `stitch_group_id` when missing.
-

`normalize_state()`

Normalizes the carry-forward state.

It ensures valid values for:

```
status
numbering_system
active_point_number
active_stitch_group_id
expected_next_main_point
last_visible_text
```

`validate_and_normalize_output()`

Takes raw model JSON and converts it into the correct output schema.

If the model returns no valid blocks, this function creates a fallback uncertain block.

`fallback_output()`

Creates a fallback output when extraction fails completely.

If the previous state was open, it assumes the current page continues the previous point.

Otherwise, it marks the page as uncertain.

run_extractor_once()

Runs the extractor for one chunk.

Flow:

1. Build prompt.
2. Call model through `llm_client.py`.
3. Parse model response as JSON.
4. Normalize output.
5. Retry on failure.
6. Return fallback output if all retries fail.

process_chunk()

Runs extraction and validation for one chunk.

Flow:

1. Run extractor once.
2. If validator is disabled, return output.
3. If validator is enabled, validate output.
4. If validator accepts, return output.
5. If validator rejects, send correction instruction back to extractor.
6. Repeat repair loop up to `MAX_VALIDATOR_REPAIR_ROUNDS`.

run_extraction_for_document()

Processes all chunks for one document.

It maintains:

- previous carry-forward state,
- previous open point block,
- page-level extraction report,

- validation logs,
 - timing information,
 - failed pages.
-

run_extraction()

Main entry point.

It processes one or more documents and saves the final Phase 2 report.

What anyone Can Safely Change

```
INPUT_CHUNKS_PATH
MODEL_BACKEND
HF_MODEL
OLLAMA_MODEL
THINK
TEMPERATURE
TOP_P
NUM_CTX
MAX_NEW_TOKENS
MAX_RETRIES
USE_VALIDATOR_AGENT
MAX_VALIDATOR_REPAIR_ROUNDS
PREVIOUS_OPEN_POINT_TEXT_LIMIT
CHUNK_START
CHUNK_END
```

Recommended full-document setting:

```
CHUNK_START = 0
CHUNK_END = None
USE_VALIDATOR_AGENT = True
```

What Should Be Changed Carefully

Be careful changing:

```
SYSTEM_PROMPT
output_schema
default_content_block()
normalize_block()
normalize_state()
fallback_output()
process_chunk()
```

If the schema changes here, `validator.py` and `stitch_points.py` must also be updated.

4.7 `validator.py`

Purpose

`validator.py` is the second LLM agent in the pipeline.

Its job is not to re-extract the page. Its job is to check whether the extractor made one of four major harmful errors.

The validator improves stability because legal extraction often fails in small ways that create large downstream stitching problems.

Major Update in Current Version

The validator no longer calls Ollama directly.

It now imports:

```
from llm_client import LLMRequestConfig, chat_completion
```

and calls the model through `llm_client.py`.

This means the validator can now use either:

HuggingFace local model
Ollama local server

independently from the extractor.

Validator Backend Config

Current validator config:

```
VALIDATOR_BACKEND = "hf"  
VALIDATOR_HF_MODEL = "google/gemma-4-E4B-it"  
VALIDATOR_OLLAMA_MODEL = "gemma4:latest"  
  
THINK = False  
VALIDATOR_TEMPERATURE = 0.3  
VALIDATOR_TOP_P = 0.2  
VALIDATOR_NUM_CTX = 32768  
VALIDATOR_MAX_NEW_TOKENS = 1024  
VALIDATOR_TEXT_LIMIT = 2500
```

Meaning:

- `VALIDATOR_BACKEND = "hf"` means it currently uses a HuggingFace local model.
 - To use Ollama, set `VALIDATOR_BACKEND = "ollama"`.
 - The validator output is small, so `VALIDATOR_MAX_NEW_TOKENS = 1024` is enough.
 - `VALIDATOR_TEXT_LIMIT` keeps previous and next page context compact.
-

Main Input

The validator receives:

```
{  
  "chunk_id": "",  
  "page_number": 0,  
  "previous_state": {},  
  "previous_tail": "",  
  "current_page_text": "",  
}
```

```
"next_head": "",
"previous_open_point_block": {},
"extractor_output": {}
}
```

Main Output

The validator returns:

```
{
  "is_valid": true,
  "error_types": [],
  "correction_instruction": ""
}
```

or:

```
{
  "is_valid": false,
  "error_types": ["wrong_block_type"],
  "correction_instruction": "Retry this chunk. Legal amendment text was classified as metadata. Treat the numbered legal content as a point block."
}
```

The correction instruction is then sent back to the extractor for a repair round.

The Four Major Errors It Checks

The validator only checks four major errors.

1. **wrong_block_split**

This means a single continuous legal point was split into multiple point blocks unnecessarily.

Example:

A point contains a table, but the extractor separates the table into another point block.

That should usually be invalid.

2. copied_context_text

This means the extractor copied text from:

```
previous_tail
next_head
previous_open_point_block
```

even though the copied text does not appear in the current page text.

This is a serious source-boundary error.

3. wrong_block_type

This means:

- legal point text was classified as metadata, ignore, or uncertain, or
- clear metadata was classified as point.

Example:

```
A numbered legal amendment is marked as metadata.
```

4. carry_state_inconsistency

This means the content blocks and carry-forward state contradict each other.

Examples:


```
continues_to_next = true, but output_carry_forward_state.status = closed
```

```
continues_to_next = true, but active_point_number is empty
```

```
active_stitch_group_id changes randomly
```

Conservative Validation Philosophy

The validator is intentionally conservative.

It should not reject outputs for:

- minor wording issues,
- harmless uncertainty,
- imperfect summaries,
- small formatting differences,
- debatable metadata classification,
- missing text unless caused by one of the four major issues.

If unsure, it should mark the output as valid.

This prevents excessive retry loops.

Main Functions

`compact_text()`

Shortens long context fields.

It keeps the beginning and ending parts of the text, separated by:

...

This helps the validator stay focused while reducing prompt length.

normalize_validator_response()

Cleans and normalizes validator output.

It ensures:

- `is_valid` is boolean,
 - `error_types` is a list,
 - only allowed error types are kept,
 - duplicate error types are removed,
 - correction instruction is limited to 700 characters.
-

parse_json_object()

Parses validator response as JSON.

If the model returns extra text, it tries to extract the JSON object.

validate_extraction_with_agent()

Main validator function.

Flow:

1. Build validator payload.
2. Build validator prompt.
3. Select validator model name based on `VALIDATOR_BACKEND`.
4. Call `chat_completion()` from `llm_client.py`.
5. Parse JSON response.
6. Normalize validator response.
7. Return validation result.

If validator execution fails, it returns valid by default:

```
{
  "is_valid": true,
  "error_types": [],
  "correction_instruction": ""
}
```

This prevents validator failure from blocking the whole extraction pipeline.

What anyone Can Safely Change

```
VALIDATOR_BACKEND
VALIDATOR_HF_MODEL
VALIDATOR_OLLAMA_MODEL
THINK
VALIDATOR_TEMPERATURE
VALIDATOR_TOP_P
VALIDATOR_NUM_CTX
VALIDATOR_MAX_NEW_TOKENS
VALIDATOR_TEXT_LIMIT
```

What Should Be Changed Carefully

Be careful changing:

```
VALIDATOR_SYSTEM_PROMPT
normalize_validator_response()
validate_extraction_with_agent()
```

If a new block type such as **paragraph** is added, the validator prompt and validation rules must be updated.

4.8 **stitch_points.py**

Purpose

`stitch_points.py` is the Phase 3 stitching script.

It reads all Phase 2 page outputs, joins content blocks that belong to the same legal point, and writes the final structured document JSON.

This is the final stage of the current pipeline.

Main Input

Configured at the top:

```
PHASE2_INPUT_PATH = Path(r"phase2_output\page_outputs_2\20160408_Finance_Act_2013")
```

This can be:

1. a single document folder containing Phase 2 page JSON files,
 2. a parent folder containing multiple document folders,
 3. a nested group folder containing document folders.
-

Main Output

Final stitched document:

```
phase3_output/final_documents/<document_name>/final_stitched_document.json
```

Per-document report:

```
phase3_output/reports/<document_name>/phase3_stitch_report.json
```

Master report:

```
phase3_output/reports/phase3_master_report.json
```

Final JSON Structure

The final document looks like:

```
{
  "contents": [
    {
      "text": "Document title or heading",
      "type": "metadata",
      "clause_number": ""
    },
    {
      "text": "Full stitched legal point text...",
      "type": "clause",
      "clause_number": "§"
    }
  ]
}
```

Important mapping:

```
Phase 2 metadata → final metadata
Phase 2 point    → final clause
Phase 2 ignore   → skipped
Phase 2 uncertain → skipped unless INCLUDE_UNCERTAIN_IN_FINAL = True
```

Important Config Variables

```
INCLUDE_UNCERTAIN_IN_FINAL = False
FINAL_JSON_NAME = "final_stitched_document.json"
REPORT_JSON_NAME = "phase3_stitch_report.json"
```

If:

```
INCLUDE_UNCERTAIN_IN_FINAL = False
```

uncertain blocks are excluded from the final document.

If changed to:

```
INCLUDE_UNCERTAIN_IN_FINAL = True
```

uncertain blocks are included as metadata.

How Stitching Works

Phase 3 uses:

```
"stitch_group_id": "point_&"
```

to identify blocks belonging to the same legal point.

It uses two dictionaries:

```
open_groups  
latest_group_index
```

open_groups

Tracks stitch groups that are currently open because the previous block had:

```
"continues_to_next": true
```

latest_group_index

Tracks the latest known final content index for each stitch group, even if the group is not currently open.

This helps recover when the model forgot to keep a group open but later says the next block continues from previous.

Main Stitching Cases

Case 1: Explicit continuation

If a block has:

```
"continues_from_previous": true
```

and its group exists in `open_groups`, the current text is appended to the existing clause.

Case 2: Recovered continuation

If a block says:

```
"continues_from_previous": true
```

but the group is not open, Phase 3 checks `latest_group_index`.

If the group exists there, the text is appended and a warning is recorded.

Case 3: Implicit open-group continuation

If the group is open but the model forgot to set:

```
"continues_from_previous": true
```

Phase 3 still appends the text and records a warning.

Case 4: New point

If there is no matching open group, Phase 3 creates a new final clause.

Duplicate Boundary Handling

Function:

```
remove_boundary_overlap()
```

This tries to prevent duplicate text when two stitched blocks overlap at the page boundary.

It compares the end of the previous text with the beginning of the current text.

Main Functions

get_document_jobs()

Finds document folders containing Phase 2 page JSON files.

It supports:

- direct document folder,
 - parent folder,
 - nested group folder.
-

load_phase2_pages()

Loads Phase 2 page outputs and sorts them by:

```
page_number  
chunk_id
```

stitch_document()

Main stitching function.

It processes all content blocks in reading order and produces:

```
{  
  "contents": [...]  
}
```

plus a detailed report.

run_phase3()

Main entry point.

It processes one or more documents, saves final JSON files, saves reports, and creates a master report.

What anyone Can Safely Change

```
PHASE2_INPUT_PATH  
INCLUDE_UNCERTAIN_IN_FINAL  
FINAL_JSON_NAME  
REPORT_JSON_NAME
```

What Should Be Changed Carefully

Be careful changing:

```
make_group_id()
append_text()
remove_boundary_overlap()
stitch_document()
```

These functions control final document correctness.

If a new block type such as **paragraph** is added, this file must be updated.

5. Recommended Running Procedure

Step 1: Put PDF in **Documents/**

Example:

```
Documents/20160408_Finance_Act_2013.pdf
```

Step 2: Run page-level OCR

Update **page_by_page_ocr.py**:

```
INPUT_PATH = Path(r"Documents\20160408_Finance_Act_2013.pdf")
INCLUDE_HEADERS_FOOTERS = False
```

Run:

```
python page_by_page_ocr.py
```

Expected output:

```
phase1_output/page_by_page_ocr/without_headers_footers/20160408_Finance_Act_2013/page_0000.json  
phase1_output/reports/page_by_page_ocr/20160408_Finance_Act_2013_report_without_headers_footers.json
```

Step 3: Run chunking

Update `chunking.py`:

```
PAGE_OCR_INPUT =  
Path(r"phase1_output\page_by_page_ocr\without_headers_footers\20160408_Finance_Act_2013")
```

Run:

```
python chunking.py
```

Expected output:

```
phase1_output/chunks/without_headers_footers/20160408_Finance_Act_2013/chunks.json
```

Step 4: Run Phase 2 extraction

Update `stateful_extraction_2.py`:

```
INPUT_CHUNKS_PATH =  
Path(r"phase1_output\chunks\without_headers_footers\20160408_Finance_Act_2013\chunks.json")  
CHUNK_START = 0  
CHUNK_END = None  
USE_VALIDATOR_AGENT = True
```

Choose backend:

For HuggingFace:

```
MODEL_BACKEND = "hf"  
HF_MODEL = "google/gemma-4-E4B-it"
```

For Ollama:

```
MODEL_BACKEND = "ollama"  
OLLAMA_MODEL = "gemma4:latest"
```

Run:

```
python stateful_extraction_2.py
```

Expected output:

```
phase2_output/page_outputs_2/20160408_Finance_Act_2013/  
phase2_output/reports/20160408_Finance_Act_2013_report.json
```

Step 5: Run Phase 3 stitching

Update `stitch_points.py`:

```
PHASE2_INPUT_PATH = Path(r"phase2_output\page_outputs_2\20160408_Finance_Act_2013")
```

Run:

```
python stitch_points.py
```

Expected output:

```
phase3_output/final_documents/20160408_Finance_Act_2013/final_stitched_document.js  
on  
phase3_output/reports/20160408_Finance_Act_2013/phase3_stitch_report.json  
phase3_output/reports/phase3_master_report.json
```

6. Backend Switching Guide

The new backend system is controlled mainly from two files:

```
stateful_extraction_2.py  
validator.py
```

`llm_client.py` should usually stay unchanged.

6.1 Use HuggingFace for Extractor and Validator

In `stateful_extraction_2.py`:

```
MODEL_BACKEND = "hf"  
HF_MODEL = "google/gemma-4-E4B-it"
```

In `validator.py`:

```
VALIDATOR_BACKEND = "hf"  
VALIDATOR_HF_MODEL = "google/gemma-4-E4B-it"
```

Required packages:

```
pip install -U transformers torch accelerate
```

6.2 Use Ollama for Extractor and Validator

In `stateful_extraction_2.py`:

```
MODEL_BACKEND = "ollama"  
OLLAMA_MODEL = "gemma4:latest"
```

In `validator.py`:

```
VALIDATOR_BACKEND = "ollama"  
VALIDATOR_OLLAMA_MODEL = "gemma4:latest"
```

Required package:

```
pip install ollama
```

Ollama server must be running.

6.3 Use Different Backends for Extractor and Validator

This is also possible.

Example:

Extractor uses HuggingFace:

```
MODEL_BACKEND = "hf"  
HF_MODEL = "google/gemma-4-E4B-it"
```

Validator uses Ollama:

```
VALIDATOR_BACKEND = "ollama"  
VALIDATOR_OLLAMA_MODEL = "gemma4:latest"
```

This can be useful if one model is better for extraction and another is faster or more reliable for validation.

7. Dependencies

General OCR and Processing Dependencies

```
pip install pdf2image pillow opencv-python numpy pandas pypdf
```

Chandra OCR

The OCR scripts expect the **chandra** command to be available.

Test with:

```
chandra --help
```

Ollama Backend

```
pip install ollama
```

Also make sure the Ollama server is running and the configured model exists.

Check models:

```
ollama list
```

HuggingFace Backend

```
pip install -U transformers torch accelerate
```

If using a gated HuggingFace model, set:

```
HF_TOKEN=<your_token>
```

or:

```
HUGGINGFACE_TOKEN=<your_token>
```

8. Current Limitations and Warnings

8.1 Paragraph extraction is not implemented yet

The current extraction schema supports:

```
point  
metadata  
ignore  
uncertain
```

It does not yet support:


```
paragraph
```

If paragraph extraction is required, update all of these files together:

```
stateful_extraction_2.py  
validator.py  
stitch_points.py
```

Required updates:

1. Add `paragraph` to allowed block types.
2. Update the extractor prompt.
3. Update `default_content_block()`.
4. Update `normalize_block()`.
5. Update validator rules.
6. Update Phase 3 final stitching.
7. Add paragraph stitching using `stitch_group_id`.

8.2 `CHUNK_START = 48` can skip pages

In `stateful_extraction_2.py`, the current setting is:

```
CHUNK_START = 48
```

This is for debugging only.

For full extraction, use:

```
CHUNK_START = 0
```

8.3 HuggingFace does not strictly

enforce JSON

Ollama supports JSON mode more directly.

HuggingFace does not enforce JSON output natively in the same way. `llm_client.py` adds stronger JSON-only instructions, and the extractor/validator also parse JSON defensively.

Still, HuggingFace models may sometimes return extra text. That is why these functions are important:

```
extract_json_object()  
parse_json_object()  
_strip_thinking_blocks()
```

8.4 Page numbering is zero-based

`page_by_page_ocr.py` starts page numbers from 0.

This means:

```
PDF page 1 → page_0000.json  
PDF page 2 → page_0001.json
```

This is not necessarily wrong, but teammates should be aware of it.

8.5 Validator is conservative by design

The validator does not check every possible error.

It only checks four major errors:

```
wrong_block_split  
copied_context_text
```

```
wrong_block_type
carry_state_inconsistency
```

This prevents overcorrection, but it also means some minor extraction errors may pass.

8.6 Some comments are outdated

In `stateful_extraction_2.py`, one section still has a heading saying:

```
OLLAMA CALL
```

But the function is now:

```
call_model()
```

and it supports both HuggingFace and Ollama through `llm_client.py`.

This is not a functional bug, but the comment should eventually be renamed to:

```
MODEL CALL
```

9. Safe Modification Checklist

To process a new PDF

Change:

```
INPUT_PATH
```

in `page_by_page_ocr.py`.

Then update:

```
PAGE_OCR_INPUT
```

in `chunking.py`.

Then update:

```
INPUT_CHUNKS_PATH
```

in `stateful_extraction_2.py`.

Then update:

```
PHASE2_INPUT_PATH
```

in `stitch_points.py`.

To change OCR settings

Edit `page_by_page_ocr.py`:

```
METHOD  
MAX_OUTPUT_TOKENS  
TIMEOUT_SECONDS  
INCLUDE_HEADERS_FOOTERS
```

To change chunk context size

Edit `chunking.py`:

```
TAIL_SIZE
```

```
HEAD_SIZE
```

To change extractor model

Edit `stateful_extraction_2.py`:

```
MODEL_BACKEND  
HF_MODEL  
OLLAMA_MODEL
```

To change validator model

Edit `validator.py`:

```
VALIDATOR_BACKEND  
VALIDATOR_HF_MODEL  
VALIDATOR_OLLAMA_MODEL
```

To run the whole document

Edit `stateful_extraction_2.py`:

```
CHUNK_START = 0  
CHUNK_END = None
```

To run only some chunks

Edit `stateful_extraction_2.py`:

```
CHUNK_START = 10
CHUNK_END = 20
```

This runs chunks 10 to 19.

To disable validator

Edit `stateful_extraction_2.py`:

```
USE_VALIDATOR_AGENT = False
```

This makes extraction faster but less reliable.

10. Developer Notes

10.1 Schema consistency is critical

The pipeline depends on this Phase 2 structure:

```
{
  "content_blocks": [
    {
      "type": "point",
      "point_number": "",
      "text": "",
      "continues_from_previous": false,
      "continues_to_next": false,
      "stitch_group_id": ""
    }
  ]
}
```

If this schema changes, update:

```
stateful_extraction_2.py
validator.py
stitch_points.py
```

together.

10.2 `llm_client.py` should remain stable

Normal teammates should not need to edit `llm_client.py`.

They should change model settings from:

```
stateful_extraction_2.py
validator.py
```

Only edit `llm_client.py` when adding or changing model backend behavior.

10.3 Full-document run checklist

Before running the full pipeline, check:

```
INCLUDE_HEADERS_FOOTERS = False
CHUNK_START = 0
CHUNK_END = None
USE_VALIDATOR_AGENT = True
```

Also confirm the selected backend:

```
MODEL_BACKEND = "hf"
```

or:

```
MODEL_BACKEND = "ollama"
```

and in `validator.py`:

```
VALIDATOR_BACKEND = "hf"
```

or:

```
VALIDATOR_BACKEND = "ollama"
```

11. Summary for Teammates

This project converts legal PDFs into structured JSON through a multi-stage OCR and LLM extraction pipeline.

Use the scripts in this order:

```
python page_by_page_ocr.py
python chunking.py
python stateful_extraction_2.py
python stitch_points.py
```

The role of each main file is:

<code>page_by_page_ocr.py</code>	→ OCR each PDF page and save page JSON files.
<code>chunking.py</code>	→ Convert page JSON files into chunks.json.
<code>llm_client.py</code>	→ Shared model backend for HuggingFace/Ollama.
<code>stateful_extraction_2.py</code>	→ Extract legal points, metadata, ignore blocks, and uncertain blocks.
<code>validator.py</code>	→ Check extractor output for major harmful errors.
<code>stitch_points.py</code>	→ Stitch multi-page points into final JSON.

Final output:


```
phase3_output/final_documents/<document_name>/final_stitched_document.json
```

Most important full-run reminder:

```
CHUNK_START = 0
```

If this is not set to 0, the extractor may skip earlier chunks.

The newest architectural change is `llm_client.py`. It makes the extraction and validation system backend-flexible, so the project can now switch between HuggingFace local models and Ollama without rewriting extractor or validator logic.